

Adidas US Sales Interactive Dashboard

MSBA Group Project Report

Pepperdine University — Graziadio Business School

Team CJS — Cayden, Jules, Sophia

1. Problem Statement and Objective

Problem Statement: Retail businesses such as Adidas require timely, data-driven insights to monitor sales performance across regions, retailers, product lines, and sales channels in order to remain competitive and allocate resources effectively.

Objective: To design and develop an interactive, multi-visualization business dashboard using Python, Streamlit, and Plotly that enables business users to explore Adidas US sales data (2020–2021) across multiple dimensions through dynamic filtering.

2. Pseudocode / Flowchart Design

The following pseudocode outlines the complete dashboard structure, layout, and each visualization's logic including feature enhancements.

BEGIN DASHBOARD

1. LOAD DATA

- Import libraries: pandas, plotly, streamlit, numpy
- Read 'Adidas US Sales Datasets.xlsx' into dataframe
- Identify correct header row containing 'Retailer'
- Convert Invoice Date to datetime format
- Extract Month and Year fields for time-based filtering
- Convert sales, profit, and units columns to numeric
- Return cleaned dataframe

2. DEFINE LAYOUT

- Set page config: wide layout, title, icon
- Display dashboard title and subtitle
- Create sidebar with filters: Region dropdown and Year radio toggle
- Display live summary metrics in sidebar: Total Sales, Operating Profit, Units Sold

3. FILTER LOGIC

- WHEN user selects Region: IF Region == 'All' use full dataframe, ELSE filter rows where Region == selected value
- WHEN user selects Year: IF Year == 'Both' keep current filtered dataframe, ELSE further filter where Year == selected value
- All 4 visualizations receive the same filtered dataframe

4. VISUALIZATION 1 — Line Chart (Monthly Sales Trend)

- Group filtered data by Month, aggregate sum of Total Sales per month
- Plot: x = Month, y = Total Sales (blue line + markers)
- AI Enhancement: add 3-month rolling average trendline (amber dashed)
- Student Enhancement: year radio toggle filter in sidebar

5. VISUALIZATION 2 — Bar Chart (Sales by Retailer)

- Group filtered data by Retailer, aggregate sum of Total Sales and Operating Profit
- Sort descending by Total Sales
- AI Enhancement: dual y-axis overlay with Operating Profit line + red dotted average reference line
- Student Enhancement: single horizontal average reference line using `add_hline()`

6. VISUALIZATION 3 — Donut Chart (Sales by Method)

- Group filtered data by Sales Method, aggregate sum of Total Sales
- Plot: donut chart (hole = 0.4)
- AI Enhancement: explode largest slice, custom brand colors, percentage labels
- Student Enhancement: percentage labels via `textinfo` parameter

7. VISUALIZATION 4 — Scatter Plot (Units Sold vs Operating Profit)

- Use filtered row-level data, plot x = Units Sold, y = Operating Profit, color by Product
- AI Enhancement: normalized bubble sizing by Total Sales, per-product numpy regression trendlines, rich hover tooltips
- Student Enhancement: product category multiselect filter in sidebar

8. RENDER DASHBOARD

- Display all 4 charts in 2x2 grid layout using `st.columns(2)`
- All charts update simultaneously on any filter change
- Display footer caption

END DASHBOARD

3. Dashboard Development

The core dashboard was developed using GitHub Copilot via a prompt-driven programming approach. Specific, targeted prompts were submitted to Copilot for each visualization, and the generated code was reviewed, tested, and integrated into the final Streamlit application.

The dashboard was built using Streamlit as the application framework and Plotly for all interactive visualizations. The dataset used is the Adidas US Sales Dataset (2020–2021), containing 9,648 records across 13 variables including retailer, region, product, sales method, invoice date, total sales, operating profit, and units sold.

The final submission includes two Streamlit dashboards:

- `dashboard.py` — AI-assisted, fully enhanced dashboard with region filter sidebar, rolling average, dual-axis charts, bubble scatter, and exploded donut
- `student_dashboard.py` — Student-authored baseline dashboard with year toggle and product category multiselect filter

Both dashboards share the same underlying dataset and `load_data()` function, with all four visualizations updating reactively when any filter is changed.

4. Feature Enhancements

AI-Generated Enhancements (via GitHub Copilot)

Viz 1 — Line Chart

Copilot Prompt: "I have a Plotly line chart showing monthly Adidas sales over time. Suggest and implement one improvement or interactive feature to make this visualization more insightful for a business user."

Copilot Enhancement: Added a 3-month rolling average trendline overlaid on the monthly sales line in a dashed amber color, smoothing out short-term fluctuations to reveal underlying sales momentum.

Viz 2 — Bar Chart

Copilot Prompt: "I have a Plotly bar chart showing Adidas total sales by retailer. Suggest and implement one improvement or interactive feature that adds business value."

Copilot Enhancement: Implemented a dual y-axis overlay adding an Operating Profit line on a secondary axis alongside the sales bars, plus a red dotted average sales reference line with annotation.

Viz 3 — Donut Chart

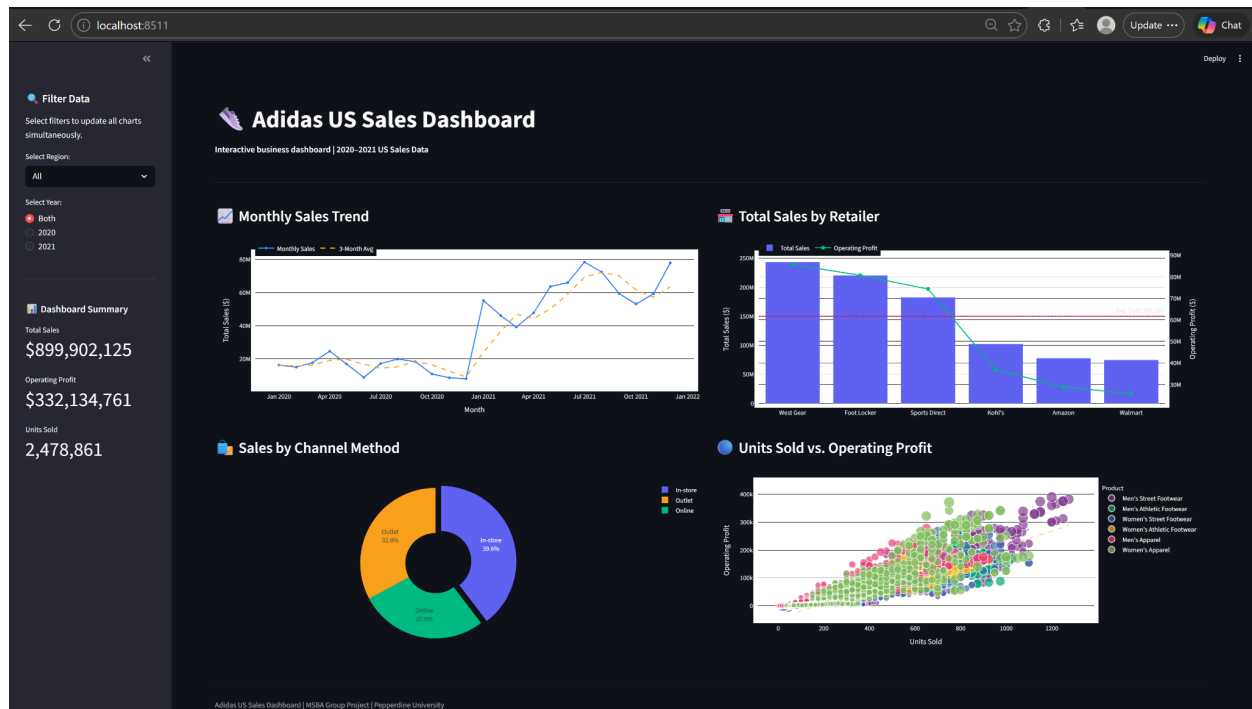
Copilot Prompt: "I have a Plotly donut chart showing Adidas sales split by sales method. Suggest and implement one enhancement or interactive feature."

Copilot Enhancement: Added slice explosion on the largest segment combined with custom brand-aligned colors and percentage labels directly on each slice.

Viz 4 — Scatter Plot

Copilot Prompt: "I have a Plotly scatter plot showing Units Sold vs Operating Profit colored by product category. Suggest and implement one improvement or interactive feature for business users."

Copilot Enhancement: Implemented per-product regression trendlines using numpy polynomial fitting, alongside normalized bubble sizing proportional to Total Sales and rich hover tooltips showing Retailer and Region.



Student-Generated Enhancements

Viz 1 — Year Toggle Filter

Added a year radio toggle (2020 / 2021 / Both) in the sidebar that filters all charts simultaneously, allowing direct year-over-year comparison across the entire dashboard.

Viz 2 — Average Reference Line

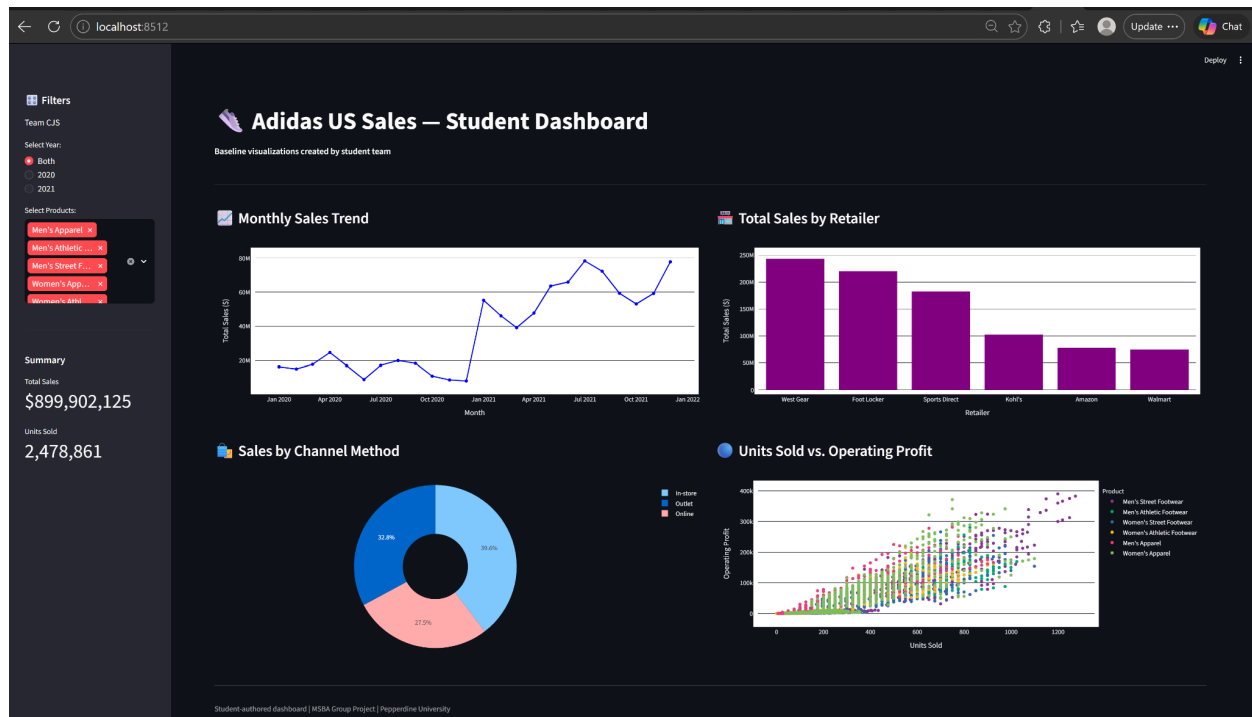
Added a horizontal reference line at average sales value with annotation, providing a simple benchmark for comparing retailer performance without the complexity of a dual axis.

Viz 3 — Percentage Labels

Added percentage labels directly on each donut slice via the textinfo parameter so business users can immediately read proportional values without hovering.

Viz 4 — Product Category Multiselect Filter

Added a product category multiselect filter in the sidebar allowing users to isolate specific product lines across all four visualizations simultaneously.



5. Comparative Evaluation

Table 1: Feature Enhancements — Technical Comparison: AI vs. Student

Visualization	AI-Generated Enhancement	Student Enhancement	Technical Comparison (Lines of Code)
Viz 1 — Line Chart	3-month rolling average trendline overlaid in dashed amber	Year radio toggle filter (2020 / 2021 / Both) applied across all charts	AI: ~18 lines; introduces pandas rolling window logic and a second Scatter trace. Student: ~8 lines; simple conditional filter with 3 radio options. AI is more complex; the student enhancement is more maintainable.
Viz 2 — Bar Chart	Dual y-axis overlay with Operating Profit line + red dotted average reference line with annotation	Single horizontal average reference line using <code>add_hline()</code>	AI: ~22 lines using <code>make_subplots</code> with <code>secondary_y=True</code> . Student: ~3 lines using <code>add_hline</code> . AI is significantly more complex; the student approach is concise and readable.
Viz 3 — Donut Chart	Exploded largest slice + custom brand-aligned colors + percentage labels on each slice	Percentage labels only via <code>textinfo</code> parameter	AI: ~12 lines including dynamic pull array via <code>idxmax</code> logic and custom color list. Student: ~6 lines. AI adds visual dynamism; students achieve the same informational goal more simply.

Visualization	AI-Generated Enhancement	Student Enhancement	Technical Comparison (Lines of Code)
Viz 4 — Scatter Plot	Per-product numpy regression trendlines + normalized bubble sizing by Total Sales + rich hover tooltips	Product category multiselect filter isolating specific product lines across all charts	AI: ~20 lines including numpy polyfit loop and bubble normalization formula. Student: ~10 lines using st.multiselect. AI is computationally heavier; student adds strong interactivity.

Table 2: Feature Enhancements — Business-Focused Comparison: AI vs. Student

Visualization	AI-Generated Enhancement	Student Enhancement	Technical Comparison (Lines of Code)
Viz 1 — Line Chart	3-month rolling average trendline	Year toggle filter	The AI rolling average reveals underlying sales momentum and is more analytically valuable for forecasting. The student year toggle enables direct year-over-year comparison and is highly practical for business users. AI wins on analytical depth; student wins on usability.
Viz 2 — Bar Chart	Dual y-axis profit overlay + average reference line	Single average reference line benchmark	The AI dual-axis chart exposes the revenue-to-profitability relationship per retailer, enabling margin analysis. The student avg line provides a simple performance benchmark. AI version provides stronger decision-support for retail partnership strategy.
Viz 3 — Donut Chart	Exploded slice + custom colors + percentage labels	Percentage labels only	Both serve the same core need — making proportions readable. AI version is more visually polished and automatically draws attention to the dominant channel. Student version achieves the same informational goal with less visual complexity. AI wins for executive presentations; student is equally effective for operational reads.
Viz 4 — Scatter Plot	Per-product regression trendlines + bubble sizing	Product category multiselect filter	AI trendlines show which product categories have stronger unit-to-profit relationships, directly informing inventory and pricing decisions. The student product filter is highly actionable for category managers needing to isolate specific lines. AI is more analytically sophisticated; student is more operationally practical.

6. Analysis and Discussion

Which approach was more efficient, with justification?

The AI-generated approach using GitHub Copilot was more efficient in terms of development speed and code sophistication. Copilot produced complex features such as dual y-axis subplots, rolling average calculations, and per-product regression trendlines in a fraction of the time it would take to manually write it. However, the student-generated approach was more efficient in terms of simplicity being how the code is shorter, easier to read, and less likely to introduce errors. For a business dashboard intended for rapid iteration and stakeholder review, the AI approach delivered greater output per unit of development time. For a dashboard intended for long-term maintenance by a non-technical team, the student approach would be preferable.

Major Challenge: Linked Visualizations

The major challenge encountered was ensuring all four Plotly visualizations rendered and updated simultaneously within the Streamlit environment. When initially prototyped in Google Colab, the ipywidgets dropdown would only display the last rendered figure rather than all four charts in a linked layout. The solution was to migrate from Google Colab to a local VS Code environment running Streamlit, which natively supports reactive re-rendering of all components on filter change without requiring widget managers or tunnel configurations. In Streamlit, any change to the sidebar dropdown or radio toggle automatically triggers a full page re-render, ensuring all four visualizations update simultaneously with the filtered dataframe — fully satisfying the linked visualization requirement.

7. Conclusion

This project successfully demonstrated the design and development of an interactive, multi-visualization business dashboard using Python, Streamlit, and Plotly applied to the Adidas US Sales dataset (2020–2021). Two dashboards were produced — an AI-assisted version leveraging GitHub Copilot for sophisticated enhancements including rolling averages, dual-axis overlays, and regression trendlines, and a student-authored baseline version featuring simpler visualizations with year-based and product-based interactive filters. The comparative evaluation revealed that AI-generated enhancements excelled in analytical depth and visual sophistication, while student-generated enhancements prioritized usability and operational clarity. Together, both approaches illustrate how human judgment and AI assistance can complement each other in data-driven dashboard development, producing a final product that is both technically robust and practically useful for business decision-making.